

Tech Stacks

[Apa itu Tech Stacks?](#)

DAFTAR ISI

DAFTAR ISI	2
Overview	1
FE & BE	1
Typescript	1
Next.JS	2
Tailwind	2
Prisma	3
NextAuth	4
AOS	4
DB	4
Headless CMS	5
Dato CMS	5
GraphQL	6
Analytics	8
Google Analytics	8
Hotjar	8

Overview

Website ini dibuat dengan bahasa [Typescript](#), FE (*front-end*) dan BE (*back-end*) dengan (still under consideration) – hosted oleh [Vercel](#) – dan DB (*database*) dengan [CockroachDB](#) (*hosted* di tempat yang sama). Selain itu, CMS (*Content Management System*) yang digunakan bersifat *headless*, yaitu [DatoCMS](#). Ada (agak) banyak *library* dan *utilities* yang digunakan untuk mempermudah pengembangan website ini dengan menyediakan semua dari *auto-documentation* sampai *type safety*.

FE & BE

Typescript

Javascript but with Types. Kita pakai bahasa ini karena [ini](#). Typescript adalah bahasa yang *easy to learn but hard to master* sehingga lebih baik untuk dipelajari sambil jalan, tetapi *knowledge* yang benar-benar mendalam tentang Typescript sebenarnya tidak terlalu diperlukan dalam kebanyakan hal yang nantinya kita lakukan. Ketahui apa itu Typescript dari [sini](#) dan lanjutkan dengan video [ini](#). Membaca-baca proyek yang dibuat dengan Typescript (contohnya [ini](#)) bersama dengan membaca [dokumentasi](#) (walau dokumentasinya agak *clunky* secara struktur dibanding kebanyakan *library* lain sekarang) ataupun Googling ketika ada yang tidak mengerti adalah langkah selanjutnya belajar Typescript.

Typescript memang memberikan *overhead* dalam proses pengembangan karena kita harus membuat tipe dulu, memastikan kode

kita tidak menyalahi aturan tipe yang ada, dll., tetapi jika kita sudah bisa menggunakan Typescript dengan baik, malah ada banyak waktu yang dihemat (*see aforementioned video*). Malah, kalau sudah biasa pakai Typescript, menggunakan Javascript akan terasa menyiksa diri.

FE Framework

Next.JS

Gold standard dari meta-framework yang sudah *ubiquitous* di industri. Kenapa kita pakai meta-framework, bukan framework saja? Karena [ini](#). [Dokumentasi](#) dari Next.JS sangat lengkap dan *helpful*. [Video](#) ini membahas dasar fitur-fitur yang ada di dalam Next.JS. Tidak terlalu banyak video tutorial mengenai Next.JS yang app router atau Next.JS 13. Dikarenakan update app router ini merupakan optimasi dari NEXT.JS sebelumnya dan masih sangat baru, sehingga lebih dianjurkan untuk membaca dokumentasi.

Tailwind CSS

Styling website dengan menggunakan CSS dapat dilakukan dengan mudah melalui Tailwind CSS. Keunggulan dari menggunakan Tailwind CSS [sangat banyak](#), dimana yang paling utama adalah DX (Developer Experience), alasan teknis seperti penghematan ukuran bundle dan dokumentasinya yang sangat lengkap dan detail.

Mempelajari Tailwind CSS tidak sulit karena memiliki dokumentasi yang sangat baik, sehingga Anda dapat dengan mudah mencari cara

untuk melakukan styling CSS yang diinginkan menggunakan Tailwind. Jika Anda merasa membutuhkan bantuan lebih lanjut, tersedia pula video tutorial yang dapat membantu Anda melihat Tailwind CSS in action. Beberapa video tersebut dapat diakses di tautan berikut [ini](#) dan [ini](#).

Prisma

Prisma merupakan sebuah ORM (Object Relational Mapper) yang akan kita gunakan. [ORM](#) adalah sebuah teknologi yang menyederhanakan proses melakukan query ke database dengan menyediakan sebuah abstraksi untuk melakukan query tersebut. Keberadaan abstraksi ini membantu membuat proses query ke database menjadi lebih mudah dan aman, menghindari masalah seperti injeksi SQL dan berbagai masalah lainnya.

Untuk lebih memahami konsep ini, dapat dilihat demonstrasi penggunaan Prisma yang memberikan penjelasan terbaik mengenai hal tersebut. Dasar dari Prisma yang mencakup hampir semua yang kita akan pakai ada dijelaskan dalam video [ini](#). Adapun hal-hal lebih khusus atau spesifik mungkin dapat dicari dan dipelajari lebih lanjut dalam [dokumentasi](#) Prisma atau Anda dapat melihat video [berikut](#).

NextAuth

NextAuth.js adalah sebuah pustaka open-source yang populer dalam ekosistem React yang digunakan untuk mengelola otentikasi

(authentication) di aplikasi web. Dengan menggunakan NextAuth.js, Anda dapat dengan mudah mengintegrasikan berbagai layanan otentikasi pihak ketiga seperti Google, Facebook, GitHub, Twitter, dan banyak lagi, serta menyediakan otentikasi berbasis email dan kata sandi. Belajar dari dokumentasi di [sini](#).

AOS

AOS adalah library animasi yang merupakan library external untuk react, tetapi tetap digunakan untuk memberikan efek animasi pada elemen-elemen di dalam website saat discrolling. Anda dapat melihat demonstrasi animasi tersebut langsung di [sini](#). Library ini mudah dipelajari dari [dokumentasi](#) di repositori GitHub-nya.

Perlu diingat bahwa meskipun library ini masih berfungsi, karena tidak diperbarui dalam beberapa waktu (terakhir pada 2018), ia masih menggunakan JavaScript murni dan tidak memiliki tipe. Oleh karena itu, tidak ada dukungan auto-completion untuk objek konfigurasi awal dari AOS.init atau jenis properti yang dapat diberikan ke elemen HTML. Selain itu, untuk menggunakannya, Anda perlu menggunakan "workaround" [berikut](#) yang sudah dikonfigurasi oleh *yours truly*.

Headless CMS

[Apa itu CMS dan headless CMS.](#)

Kita akan menaruh konten-konten di dalam website-nya ke dalam sebuah *headless* CMS. Konten-konten tersebut kemudian akan kita *fetch*

saat *build time* dengan [getStaticProps](#) dan divalidasi ulang dengan [On-Demand ISR](#) di bagian yang dibutuhkan. Kenapa kita pakai *headless* CMS dan tidak menaruh konten kita *hardcoded* di website? Karena itu bukan *best practice* dan jujur 🙄😓.

Alasan yang lebih penting lagi, sebisa mungkin kita sebagai departemen website menghindari menjadi *bottleneck* untuk penambahan konten ke website dengan membuat divisi dan departemen lain bisa menambahkan kontennya sendiri melalui *headless* CMS. Selain itu, hal trivial seperti *request* untuk memperbaiki *typo* atau menambahkan *merchandise* baru ke toko menjadi *automated* dengan menggunakan *headless* CMS ini agar kita bisa fokus melakukan hal-hal yang lebih penting.

Dato CMS

Banyak penyedia layanan *headless* CMS tersedia di pasaran saat ini. Namun, mereka memiliki dua kesamaan: jika mereka menangani hosting, biayanya sangat tinggi, dan jika layanannya *self-hosted*, maka pengguna tetap harus membayar biaya hosting. Oleh karena itu, dalam memilih *headless* CMS yang sesuai untuk konteks kita, penting untuk mencari yang menawarkan batasan *free-tier* yang memadai dan tidak *self-hosted*. Dalam kriteria ini, ada tiga *headless* CMS yang memenuhi persyaratan tersebut, yaitu Sanity, Contentful, dan DatoCMS. Di bawah ini adalah alasan mengapa dua CMS lainnya tidak dipilih.

1. Contentful memiliki pengalaman pengembang dan penulis konten yang kurang baik, berdasarkan pengalaman mencoba Contentful langsung dan juga testimoni dari orang lain.
2. Sanity memiliki batasan media yang jauh lebih besar dibandingkan DatoCMS (5 GB dibandingkan 200 MB) dan platform-nya sangat dapat disesuaikan (customizable). Namun, karena tingkat customizability ini, Sanity memerlukan lebih banyak effort untuk setup dan maintenance. Salah satu kekurangan lainnya adalah bahwa Sanity Studio (tempat penulis konten menambahkan konten ke CMS) adalah self-hosted, meskipun ini dapat diatasi dengan menggunakan Netlify untuk hosting. Namun, masih merupakan drawback (kelemahan) karena perlu usaha tambahan untuk mengaturnya. Selain itu, ada fitur seperti enumerated type yang tampaknya tidak tersedia dalam dokumentasi yang telah saya baca. Secara pribadi, 200 MB tampaknya cukup untuk digunakan, selama ada batasan ukuran yang cukup ketat untuk setiap field gambar.

Dalam memilih DatoCMS, alasan lain yang menjadi pertimbangan adalah pengalaman pengguna yang sangat baik, baik untuk pengembang maupun penulis konten.

*Note, kalau ketemu headless CMS yang menurut kalian lebih bagus dari DatoCMS, **do let me know.***

GraphQL

Untuk mengambil data dari DatoCMS, digunakan bahasa bernama [GraphQL](#) yang diarahkan ke [satu endpoint](#). Memahami dan menggunakan

bahasa query ini cukup sederhana, karena dokumentasi yang telah disediakan pada tautan sebelumnya menjelaskan dengan jelas cara melakukan query, termasuk tindakan lebih kompleks seperti penyaringan dan paginasi (gunakan call optimized). Selain itu, setiap proyek yang ada di DatoCMS memiliki API explorer yang memungkinkan kita mengeksekusi query yang kita tulis langsung ke endpoint DatoCMS, sehingga kita dapat melihat hasilnya secara langsung.

Analytics

Google Analytics

Nggak perlu dijelaskan kenapa kita pakai [ini](#) sebagai salah satu tool analytics. Cara *setup* ada di [sini](#) tetapi seharusnya sudah dilakukan juga jadi tidak perlu dipelajari lagi.

Hotjar

[This library](#) is so damn creepy but also so damn cool. Bisa lihat *heatmap* (bagian mana dari website yang paling sering dipakai) dan pengunjung spesifik melakukan apa di website-nya (seperti *screen record* dari aktivitas mereka).

CI/CD

CI/CD utamanya sudah di-*handle* oleh Vercel. Jika PR gagal *build* untuk suatu alasan maka Vercel akan menunjukkan itu di PR-nya.

Untuk menjaga konsistensi dari *code style*, kebanyakan kode *framework* yang ada – *t3 included* – menyediakan Prettier dan ESLint config *out-of-the-box* untuk bisa dipakai. Kita membawa ini *one step further* dengan mengintegrasikan *linting* otomatis ke dalam proses CI/CD menggunakan Github Actions. Dengan konfigurasi yang sudah dibuat, jika ada kode yang tidak sesuai *style*-nya dengan *config* Prettier dan ESLint yang ada maka Actions ini tidak akan langsung gagal, tetapi mencoba memperbaiki kodenya terlebih dahulu dengan memberikan *commit* ke PR terkait yang berisi perbaikan (*yes, it's really that magical*). Sehingga setelah *commit*, sebaiknya selalu *pull* sebelum lanjut bekerja karena bisa saja Actions ini meng-*push* sebuah *commit* berisi *code formatting* setelah *commit* yang kita buat tersebut. *Regardless*, kita tetap **wajib** memasang ESLint dan Prettier di dalam VSCode untuk memformat kode kita.